# Sistemi operativi

| | |
|---|---|
| Created | @5 de marzo de 2026 21:36 |
| Class | sistemi operativi |
| Lia&Simo | Ⓢ SIMONA LI CALZI Ⓘ lia🦋 |

## Info importanti

Esame: teoria(65%) e lab(35%).  la teoria e la pratica si possono fare in appelli diversi. la teoria è sia test che orale. la prova di teoria se superata scade a maggio dell'anno successivo. , Il test è a risposta aperta ci sono anche esercizi.

link:https://diraimondo.dmi.unict.it/teaching/sistemi-operativi/

le slide (4/5 unità vengono caricate quando si finisce l'argomento) : https://git.crypto.dmi.unict.it/teaching/operating-systems.2024-2025 (ci sono quelle dell'anno scorso).

# Lezione 5 Marzo

## Cos'è un sistema operativo?

Un moderno calcolatore è formato da: uno o più processori, memoria centrale, dischi, stampanti e altre periferiche I/O.

Per gestire tutte le componenti serve uno strato intermedio software: il Sistema Operativo.(Sistema operativo software che gestisce l'hardware)

I processi sono dei programmi che una volta mandati in esecuzione diventano un'entità attiva. Nei sistemi multiprogrammati troviamo più processi.

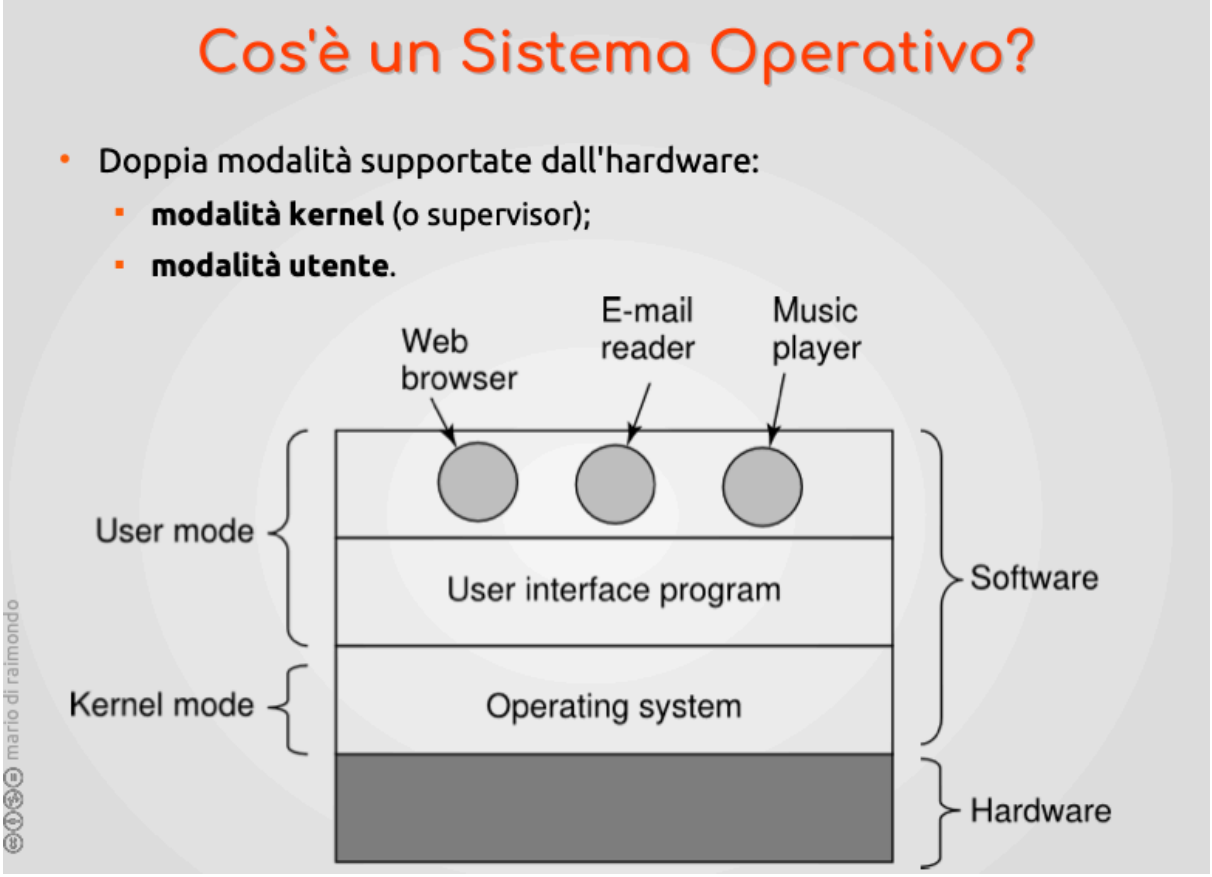
-Programma: entità statica

-Processo: entità dinamica

-Multiprogrammazione: capacità del sistema operativo di gestire più processi.

Qualunque elaboratore che ha delle risorse da gestire ha un sistema operativo, anche se non  interagiamo direttamente con esso ma con delle applicazioni.

Tra i processi e l'hardware c'è questo strato software che possiamo indicare come sistema operativo, che può essere visto da due punti di vista. Punto di vista basso (hardware) quindi il SO è un gestore di risorse; punto di vista alto(utente) chi utilizza un dispositivo che in realtà interagisce con le applicazioni.



Modalità di esecuzione:

📌

1. Kernel Mode (Supervisor Mode): È una modalità privilegiata. Il codice in esecuzione nella CPU può eseguire tutte le azioni previste dalla CPU.

Può eseguire:

**Manipolazione diretta della memoria** (allocazione, protezione, mappatura degli indirizzi

fisici)

Accesso diretto all'hardware e ai registri di controllo

Gestione delle interruzioni e delle eccezioni

Configurazione della MMU (Memory Management Unit)

I processi eseguiti in modalità kernel vengono definiti **processi multiprogrammati:** possono avviare autonomamente altri programmi e/o sottoprocessi secondo quanto stabilito dalla loro logica interna.

I processi lavorano in spazi separati tra di loro, il Sistema Operativo agisce come un'entità separata e superiore che arbitra l'accesso alle risorse.

💡

Il **Kernel** è il nucleo del Sistema Operativo. Non è hardware, è software. A differenza dei normali programmi, il kernel è una componente **residente in memoria** (viene caricato nella RAM all'avvio del sistema e da quel momento in poi ha il controllo diretto dell'hardware)
Il kernel, in particolare, è progettato per operare in maniera **parallela e compartimentata**.

Questa caratteristica è fondamentale per due ragioni principali:

1. **Efficienza del sistema**: consente di gestire più richieste contemporaneamente senza colli
di bottiglia.

2. **Isolamento dei conflitti**: processi che competono per le stesse risorse vengono arbitrati
dal kernel, evitando stati inconsistenti o deadlock.

Il codice in esecuzione nella CPU ha dei limiti, ci sono delle istruzioni privilegiate e non privilegiate.

📌

2. User Mode: È una modalità **non privilegiata**. (ES. riprogrammare il controller della memoria o accedere direttamente al disco.)

Quando un'applicazione ha bisogno di una risorsa hardware (es. leggere un file), non può farlo da sola. Deve invocare una **System Call** (chiamata di sistema). In quel momento, la CPU effettua un "trap" (un'interruzione software), passa dalla modalità utente alla modalità kernel, esegue il compito richiesto in modo sicuro e poi torna alla modalità utente.

È importante non cadere nell'errore di credere che *tutto* il codice eseguito in modalità utente sia completamente estraneo al kernel.

In pratica, gli utenti interagiscono con il sistema tramite interfacce grafiche (**GUI — Graphical User Interface**), le quali consentono di compiere operazioni tipicamente riservate al sistema operativo in modo accessibile e sicuro, senza necessità di interagire direttamente con il kernel.

**Chiamate di Sistema (System Call)**

💡 Una system call (chiamata di sistema) **è il meccanismo fondamentale tramite cui un programma in esecuzione (user mode) richiede un servizio al kernel del sistema operativo, che opera con privilegi elevati (kernel mode)**.

**Esempio: Apertura di un file**

Quando un utente (o un programma) richiede l'apertura di un file, si innesca la seguente:

1. **L'applicazione** esegue una serie di operazioni predefinite (es. chiamata a open() in C).

2. **La CPU passa in modalità kernel**.

3. **Il kernel** esegue una serie di controlli e operazioni:
   Verifica l'esistenza del file nel filesystem.

   Controlla i permessi di accesso dell'utente richiedente
   Alloca le strutture dati necessarie
   Restituisce il controllo all'applicazione con il risultato

Il passaggio da modalità utente a modalità kernel e ritorno avviene in maniera trasparente per l'utente finale, ma è un meccanismo fondamentale per garantire la **sicurezza** e l'**integrità** del
sistema.

## Astrazione

Il sistema operativo astrae l'interfaccia, usa l'hardware e offre ai programmi applicativi una propria interfaccia che sia ad alto livello, più comoda da gestire.

Uno dei concetti chiave nell'architettura dei sistemi operativi è quello di **astrazione**. Il kernel opera a un livello di programmazione molto basso, a stretto contatto con l'hardware. Operazioni che sembrano banali nascondono in realtà una complessità considerevole.

L'obiettivo dell'astrazione non è elencare queste complessità, ma **nasconderle**. Il kernel espone all'utente (e ai programmi applicativi) un'interfaccia semplificata — un insieme di istruzioni ad alto livello — che permette di eseguire operazioni complesse senza dover conoscere i dettagli implementativi dell'hardware sottostante. Questo rende il sistema più usabile e portabile.

## Cpu e memoria virtuali

Su un protocollo posso avere una o più CPU, nel caso più semplice ne abbiamo una sola, sappiamo che si possono eseguire più processi contemporaneamente e questo può essere visto in tanti modi , ma anche come un'astrazione in cui il Sistema Operativo crea delle **CPU VIRTUALI**. Fa credere ad ogni processo  di avere una propria CPU ma in realtà sta gestendo le n CPU virtuali con l'unica fisica che ha. Questo viene chiamato **time sharing.**

La **Memoria Virtuale** è un'astrazione fondamentale fornita dal Sistema Operativo che permette a ogni processo di operare in un ambiente di lavoro isolato e apparentemente illimitato.

L'astrazione non è solo una semplificazione ma viene utilizzata dal SO per rendere l'hardware utilizzabile.

- **Approccio Bottom-Up:** Il SO è un **Resource Manager**. Gestisce la complessità dei segnali elettrici e dei registri hardware, offrendo in cambio un'interfaccia pulita (File, Processi).
- Gestire una risorsa significa arbitrare i conflitti(più processi potrebbero voler accedere ad uno stesso file), fornirne un utilizzo ordinato e coordinato.

## Il sistema operativo come gestore delle risorse

- Da un moderno sistema operativo ci aspettiamo che gestisca:
  - **più programmi** in esecuzione;
  - **più utenti**.
- Necessita **allocazione ordinata e controllata** di risorse quali: processori, memoria, unità di I/O,...
- Non solo hardware: file, database,...
- **Multiplexing**:
  - **nel tempo**: CPU, stampante,...
  - **nello spazio**: memoria centrale, disco,...

mario di raimondo

# Multiplexing

Un sistema operativo moderno deve essere in grado di gestire:

**Più programmi in esecuzione contemporanea**
**Più utenti connessi simultaneamente**

Per farlo in maniera ordinata e controllata, si ricorre al concetto di **Multiplexing**.

Strategie di gestione

1. MULTIPLEXING NEL TEMPO: Si usa per risorse che non possono essere divise fisicamente (CPU). Passare da un processo P1 , all'altro P2, significa che il SO deve fare sia un lavoro di salvataggio (i dati dell'unica CPU fisica che ho) e di ripristino(ricaricare gli stessi valori lasciati nei registri della CPU, sostituire P2 con P1).

Il multiplexing nel tempo permette di creare CPU virtuali.

2. MULTIPLEXING NELLO SPAZIO: Si usa per risorse che possono essere spezzettate (RAM). Rientra in questo caso tutto quello che riguarda degli

spazi di memoria da gestire. Diversi processi occupano contemporaneamente porzioni differenti della risorsa, il Sistema Operativo divide la RAM in blocchi e li assegna ai processi dove c'è spazio, garantendo l'**isolamento**.

Il multiplexing nello spazio permette di creare l'astrazione della memoria virtuale.

> 💡 **Perché è fondamentale? Senza multiplexing, ogni programma dovrebbe attendere che tutti gli altri terminassero prima di poter ottenere l'uso della CPU o della memoria. I sistemi moderni sarebbero di fatto inutilizzabili.**

- **Thread:** Sono flussi di esecuzione multipli **dentro lo stesso processo**. Condividono lo stesso spazio di memoria (stesse variabili, stessi file aperti) ma hanno registri e stack separati.

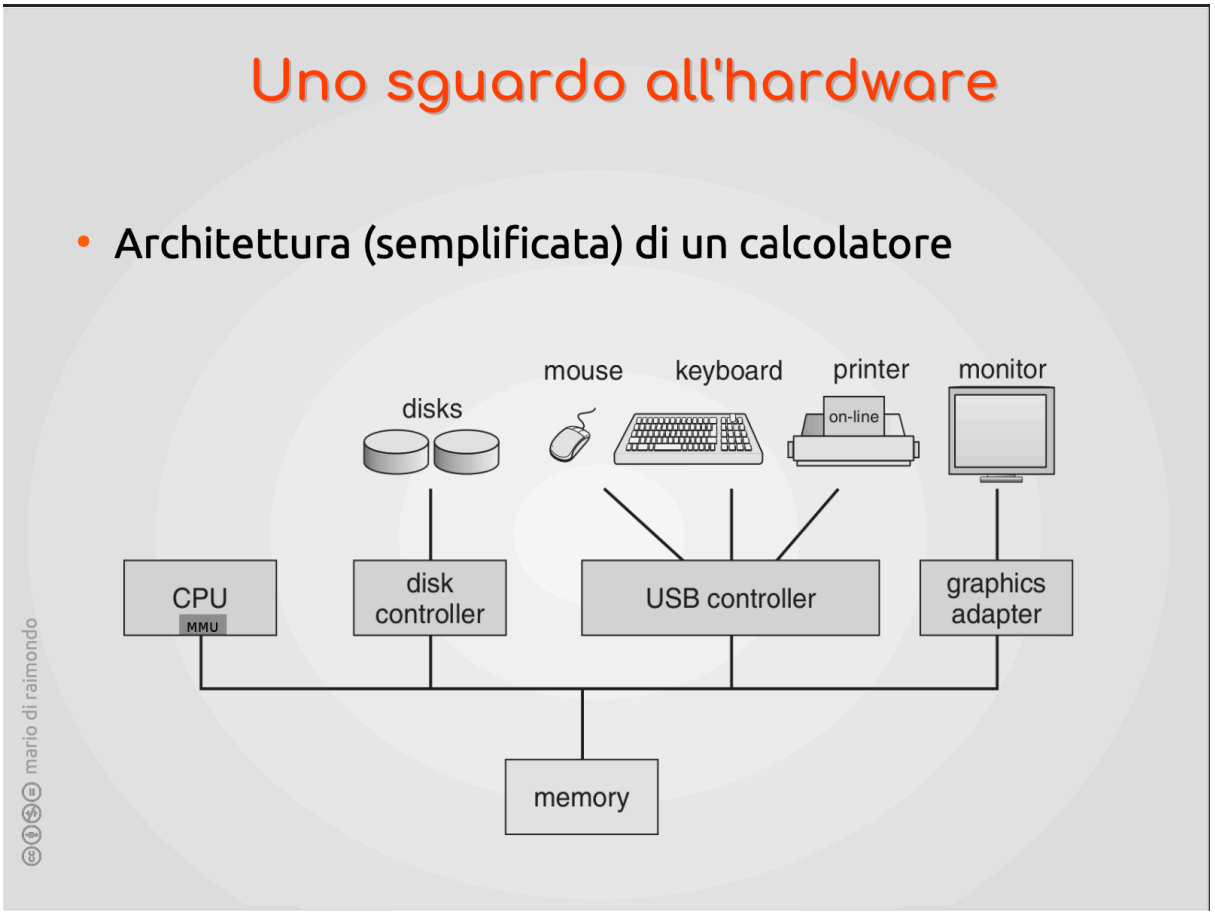
In un programma multi-thread : il primo thread esegue una procedura , il secondo un' altra  ecc.. ma tutto questo avviene nello stesso ambiente.

**Flusso di Esecuzione (Execution Flow)**

Il **flusso di esecuzione** descrive la sequenza ordinata di istruzioni che la CPU elabora per
portare a termine un programma. In un sistema multiprogrammato, questo flusso non è lineare:
il sistema operativo può interrompere l'esecuzione di un processo (tramite un'**interruzione** o allo scadere del suo *time slice*), salvarne lo stato nel **PCB** (Process Control Block), e riprendere l'esecuzione di un altro processo dallo stesso punto in cui era stato sospeso.

Questo meccanismo, noto come **context switch**, è ciò che rende possibile l'esecuzione parallela apparente su un singolo processore.

# Il Calcolatore




Un calcolatore (in maniera generica o astratta)è caratterizzato dalla presenza di una o più CPU, ogni CPU è collegata alle periferiche tramite BUS, solitamente specializzati, che ottimizzano la comunicazione tra CPU e le varie componenti periferiche dell'hardware.

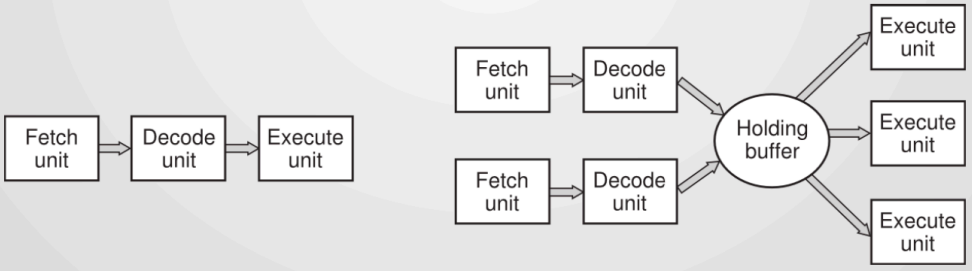
Le componenti da attenzionare sono:

RAM, memoria volatile(limitata ma veloce e direttamente accessibile dalla CPU).

## Processore

# Il Processore

- **Ciclo di base**: prelevamento (fetch), decodifica, esecuzione
- Registri particolari:
  - **Program Counter** (PC);
  - **Stack Pointer** (SP);
  - **Program Status Word** (PSW).
- Progettazioni avanzate: **pipeline**, **cpu superscalare**
  - non del tutto trasparenti al SO




mario di raimondo

Il processore è un'unità capace di unire codice, rilevato dal classico ciclo di fetch, decode, execute:

preleva dalla RAM la prossima istruzione da eseguire;

la decodifica, capendo qual è la sua semantica e gli operandi necessari;

nel momento in cui viene decodificata poi viene eseguita con operazioni logiche, aritmetica o altro...

La **RAM** funge da memoria primaria ad alta velocità per i dati e le istruzioni in uso. Per accellerare ulteriormente l'accesso, utilizza anche la **cache**, posizionata vicinissima al processore.

Dei registri generici della CPU ce ne occorrono un numero limitato, di appoggio per i dati provenienti dalle varie operazioni svolte nella CPU; ma ci sono anche dei "registri specifici" per la gestione del ciclo fetch, decode ed execute si chiamano UNIT.

Il **Program Counter** è un registro che indica alla CPU qual è l'indirizzo della prossima istruzione da eseguire. o prende l'indirizzo successivo, o se siamo in

caso di SALTO va a reimpostare l’indirizzo successivo. Il ciclo inizia proprio con l’istruzione localizzata esattamente nel PC.

Le esecuzioni effettuate nella CPU vengono affiancate da una struttura esterna, lo stack.

Lo stack di esecuzione di un programma è gestito come se fosse una pila, push e pop sempre sulla testa. In questa memoria di appoggio alloco variabili(chiesto di istanziare variabile di nome A nella memoria) e viene inserito nello stack tramite un push, questo permette che quando la procedura si chiude(quando la routine dove avevo dichiarato A si chiude) vengono fatti dei pop che deallocano quelle locazioni di memoria allocate nella funzione.  A potrebbe anche essere una procedura che ha argomenti( all’interno dello stack vengono memorizzati anche l’indirizzo di ritorno e gli argomenti x e y e inseriti nel frame di attivazione che sto creando in locazioni prefissate)

alla fine della procedura, sa quanti elementi deve deallocare e quali sono i valori di ritorno. Ciò accade anche quando una procedura A chiama una procedura B. se A chiama B allora creo frame di attivazione per A e per B e anche per altri successivi che siano C,D ecc.. ma attenzione! il valore di ritorno di C sarà B, di B sarà A e cosi via...

tutto questo funziona anche per la ricorsione, infatti si utilizza uno stack per darle vita. Per crearlo, devo avere spazio in memoria e un puntatore alla testa(memorizzato nello **Stack Pointer**).

Lo **Stack Pointer** è un registro della Cpu che si occupa di portare la CPU nella testa dello stack, registro importante che deve essere presente e lavora a stretto contatto con CALL, RETURN...

(allocazione con new non va nello stack ma nell’heap)

Altro registro importante è il Program Status Word **(PSW)**, o registro di stato, **è un registro hardware fondamentale della CPU che memorizza l’esito dell’ultimo compare attuale di un programma in esecuzione**. Contiene bit di flag che indicano i risultati delle operazioni (es. zero, riporto, overflow) e bit di controllo per la gestione degli interrupt e i privilegi.

La PSW è un'area di memoria o un registro hardware che racchiude lo stato istantaneo del processore. Quando si verifica un evento inaspettato o una richiesta esterna, il contenuto della PSW viene salvato, permettendo al processo interrotto di riprendere l'esecuzione esattamente da dove si era fermato.

w

**Componenti tipici:**

- **Condition Codes (CC):** Risultati di operazioni (es. zero, overflow, riporto).
- **Modalità di Privilegio:** Indica se la CPU opera in modalità "Kernel/Supervisore" o "Utente".
- **Abilitazione Interrupt:** Bit che attivano o disattivano la ricezione di interrupt esterni.
- **Program Counter (PC):** Spesso la PSW include o lavora in tandem con il PC, che contiene l'indirizzo della prossima istruzione da eseguire

💡

- **Kernel Mode (Supervisore):** La PSW ha un bit impostato che permette l'esecuzione di istruzioni privilegiate (accesso diretto all'hardware, gestione della memoria).

💡

- **User Mode (Utente):** La PSW indica una modalità a privilegi limitati. Se un programma utente cerca di eseguire istruzioni critiche (es. I/O), la CPU verifica la PSW e genera un'eccezione (trap).

PSW è il registro che permette il **Context Switch** (cambio di contesto):

- **Flag di Condizione:** Fondamentali per le decisioni logiche (se il flag `Zero` è attivo dopo un `compare`, allora le due variabili erano uguali).
- **Protezione (Kernel vs User Mode):** Questo è il pilastro della sicurezza informatica moderna. Impedisce a un normale programma (User Mode) di "distruggere" il sistema operativo, limitando l'accesso diretto all'hardware

CPU superscalare

# Lezione 10 Marzo

## ripassino:

utenti del sistema operativo sono i processi(system call); librerie collezioni di istruzioni precompilate che possiamo utilizzare, funzioni di libreria: codice utente che chiama altro codice utente.

chiamate di sistema appaiono allo stesso modo, ma invochiamo un servizio offerto dal sistema operativo , eseguito in modalità kernel. TRAP permette di saltare all'interno del sisyema op andando a cedere il controllo della cpu al kernel. è il compilatore che svolge il saltino al kernel per effettuare tutto il codice. pero non specifico a quale indirizzo saltare.

viene salvato nello stack tutto ciò che è utile nel ritornare. IL PC viene inserito nello stack, anche la PSW(memorizza la modalità attuale utente in questo caso), la trap permette di saltare ad una procedura prestabilita del kernel, cioè l codice implementato della funzione. Prima della trap la cpu era in mod utente, subito dopo la trap si trova in mod kernel, orami il codice è nel kernel ed è sotto il controllo del sistema operativo, che implementa tutto quello che sta dietro la richiesta e implementa la funzione(la chiama OPEN). cosi posso ritornare al nostro editor, tramite un return ritornando nell'indirizzo che era nello stack, avendo cura di tornare alla modalità utente limitata.

mod utente poi mod kernel e poi utente.cosi qualunque processo può fare richiesta al sistema operativo che poi sceglierà se soddisfarla o no.(b binario)

processo simile sono interrupt hardware, meccanismo che permeyye di notificare qualcosa alla CPU, interrompe quindi quello che sta facendo in quel momento per eseguire la routine associata all'interrupt. La CPU salva il minimo indispensabile per tornare a riprendere quello che stava facendo la CPU e viene salvato il PC, PSW e stack corrente, dopo in modo simile alla TRAP,  alll'interno della memoria esiste una tabella dove ci sono indirizzi di memoria associati ad ogni interrupt. Hardware ha cura di salvare PC PSW e di saltare alla procedura esatta dell'interrupt. Quando ha terminato, deve ritornare alla situazione precedente. per l'applicazione un interrupt non deve procurare problemi, tutto poi deve continuare normalmente.

Ogni kernel ha una funzione per eseguire un interrupt e quindi questo codice deve essere sempre eseguito quando ce una interrupt, in modalità kernel. alcune interupt si hannoa nche con divisione per zero, l'istruzione scatena l'evento (evento sincrono) (asincrono quando l'evento è scatenato dall'esempio), se l'evento è un errore fatale il processo viene terminato o anche un'istruzione illegale è impossubile da interpretare.

slide 13

tra i vari meccanismi possiamo trovare multithreading, si riferisce ad ottimizzazoni di una CPU. questo escamotage prevede di implementare un doppio contesto di esecuzione. CPU unita di elaborarzione con registri, a cui può acccedere. l'idea è: nel ciclo di decode execute ci sono tempi morti(quando la cpu deve accedere ad allocazioni vuote della ram), ho un doppio set di registri e in ognhno vengono caricati il set di valori associati a due processi: se ho il processo P1 che implica un certo numero di cicli affinche un suo fetch venga espletato,  nel tempo morto verranno eseguiti degli step del P2. puo essere vista come una complicazione, ma oermette di lavorare su contesti diversi in modo più efficace, ma non ce parallelismo, non ce una doppia CPU. in un dato istante si attenziona o uno o l'altro processo.

E come se avessi due Cpu, ma il sistema operativo deve tenere in conto che qui non ne ho due, è l'harware che finge di averne due. perche il so deve distinguere le due situazioni? puo portare a utilizzi strani della risorsa. Supponiamo di avere 2 CPu ma per oguna ho due set diu registri, quatro regstri che vengono esposte come 4 cpu, ma il sistema operativk deve capire che non sono davvero 4 CPU PERCHE ad esempio avendo solo due provessi, li carico nei primi due registri ma su due cpu distinte, scelta pessima sarebbero prendere due registri entrambe nella stessa cpu. quindi non sto sfrutttando il fatto che ho due cpu.

se ho 4 cpu ho 4 contesti e il so decidera quale dei processi caricare. perche mettere piu cpu in un sistema? per andare piu veloci. non è utile aumentare di una sola cpu i gigahertz. per aumentare la produttività e buono ottimizzare le pipeline, di complicare un po'  la pipeline ma avere piu unita di elaborazione cosi da aumentare l'efficienza.

se ho un unico pc con unica ram e 4 cpu, tutto condiviso, il che è un vantaggio da un punto di vista economico, lo svantaggio sta nel fatto che no0n è un 4x netto, perche appunto potrebbero esserci accodamenti o tempi morti, collo di bottiglia. ce differenza tra multicpu e multicore, in un sistema multicpu ha un chip distinto per ogni unita di elaborazione nella motherboard. un approccio piu recente è usare un chip unico e all'interno ha piu unita di elaborazione. (questione ingegneristica che ha dei vantaggi). ha dei vantaggi perche comunicazione tra core e core è piu veloce, vantaggi ancge sul dispendio di energia e calore dissipato infatti molto utilizzxato sui sistemi embedded, sui nostri telefoni abbiamo quetso oer ridurre consumo di energia. dal punto di bista del so, sono praticamente la stessa cosa, ci interessa il numero piu che la tecnica(multicpu e multicore sono la stessa cosa).

Parlanfo di elaborazione parallela si parla di GPU, connotate dalla presenza di unita di elaborazione piu di quelle delle cpu, ma a differenza delle cpu non ci soffermiamo perche il so non mette bocca, scenario che va al di fuori del sistema operativo. viene demandata alle applicazione che usa questo specifico meccanismo.

slide 14

Memorie che possiamo trovare: dalla rtipologia piu efficiente ma meno economica a quella meno efficiente ma piu economica. Partiamo dai registri, memoria velocissima e lavora senza ritardi e latenze rispetto al funzionamento della cpu, pero ne limita la capienza e disponibilita essendo cosi veloce. con i registri ci fareu poco se non mi appoggiassi alla RAM, ha dimensioni superiori, sono entrambe volatili, la RAM veloce si ma non quanto i registri. Abbiamo la cache anche, è una memoria ibtermedia tra CPU e RAm stessam, e va a mettere una pezza in term,ini  di efficienza sui colli di bottiglia poiche l'esecuzione di un codice fa riferimento alle stesse locazioni di memoria. avere una memoria intermedia piu veloce della ram ma meno della cpu è utile. La cache è prganizzata per linee di cache, d 64 BYTE, e una linea viene portata dentro quando la cpu fa riferimento nel breve periodo ad una parte della linea di cache, viene copiata dalla ram e portata alla cache. se presenta la word viene gestito rapidamente, caso di cache hit. caso cache miss, quando ho consukrato la cache ma non ce nessuna word dell'area che mi interessava, e quindi viene richiesta alla memoria centrale e quindi verra prelevata piu lentamente, poi depositata nella cache. ci sono algoritmi specifici, eliminoq ualcossa in cache affinche non mi crei problemi. Posso avere piu livelli di cache, e devo anche capire dove collocarli. la cache piu vicina alla cpu(livello 1) è piccola e lavora a stretto comntatto a latenza. nulla, ma è molto costosa. le cache di livello successivo nei sistemi multicore possono essere implementati in chiave diversa. potrebbero essere sia dedicate(ogni cache ha il suo livello 2) oppure è condivisa. la scelta migliore non c'e ma ci sono implicazioni diverse. il fatto di avere unica cache puo richiedere un controllo della cache piu complocato, un sistrma con un piu alto numero di core potrebbe essere ancora piu complicato. in realta il fatto di avere una sola cache puo diventare bottleneck. l'altro punto di vista non si ostacolano. impressione ad alto livello che nasconde dei prooblemi non immediati nella gestione delle cache. tutte quetse cache fanno riferimento ad un unico bus. se supponiamo di aver una linea di cache che ha una word e che quetsa area viene considerata sia sul thread 1 che sul thread 2. nel lavorar su quetsa area di memoria, le cache del secondo livello dei due core, faranno una doppia coppia copia del 2 livello di cache nel core 1 e nel

core 2. questo è un problema quando vado a scriverci sopra, se la core 2 fa una modifica deve sincronizzarsi con la core 1.

ram volatile, se spengo il pc si svuota(approfondire)

dischi magnetici e ottici ne parleremo successivamente, ci fa vedere disco elettrometìccanico: dischi concenbtrici e sono magnetizzabili e possuamo svrivere e leggere stati con una testina, una per ogni disco, per posizionarsi in una posizione possibile in un ciindro; sono le aree raggiungibili lacian do la testina li dovè. (cercalo su chat come funziona)

slide 15

connessi al controller del dischi, ad es svheda grafica è un controllo del display.

un controller eè un mini calcolatore dotato di una propria unita di elaborazione e amche memoria di buffer e input e output, fisicamente collegato al nostro disco nell'esempio. di solito il controller deve pilotare il disco, permettono di inviare comandi. Un esempio di interfaccia sono dei comandi che permettono di azionare il motore se è spento, verificare dove si trova la testina ecc. Comandi semplici e primitibi. Il controller fa da tramite tra sistema operativo e dischi.

Quando si parla di muktiprogrammazione, la cpu non mi permette di avere una determinata precisione. quindi il disco è un piccolo calcolatore che fa da se.

il suo utente è il sistema operativo. il controller espone una propria interfaccia. ogni controller deve fornire delle primitive ad alto livello, molto più semplice.

cercare mappatura in memoria come funziona caricando slide 15

viene inserito un driver dal produttore, qualuque driver per un controller di dischi deve implementare interfaccia comune, e devono implementare tipiche cose che il so chiederebbe. Questa richiesta ad alto livello viene interpretata dal driver, che conosce entrambi i linguaggi ad alto e basso livello. Il driver dialoga con il controller attraverso porte di input e output,l'interfaccia proprietaria è composta dalle cose da leggere e scruvere, per istruire il controller se leggo il manuale del dialetto del controller ci saranno tutte le spiegazioni.

mappatura I/o in memoria: il so tramite istruzioni privilegoate  puo istruire il controller dell' MMU(memory management unit) vedremo che fa da tramite  e assllver questa richiesta. si occupa di gestire le richioeste. orevede di far mapare determinate richieste su determinate porte di controller. CONTROLLARE escamotage per eseguire driver in modalita utente

La mappatura fatta una tantum è privilegiata, ma leggere e scrivere na volta fatta la mappatura non è privilegiato

slide 16

se parliamo di storage la richiesta potrebbe venire da un utente o dal kernel stesso(esigenza di leggere on scruvere sul disco) a prescindere dakk'origine, si accodano un certo numero di richieste. se la richiesta proviene da prcesso utente, la catena deve compiere il suo lavoro. se si parla di input o output, questa puo essere uan richiesta lenta. (piu tempo della cpu). se il mio editor ha richiesto di leggere un documento, l'editor dal punto di vista logico è bloccato fino a quando non avrà una risposta. non è possibile che la mia cpu rimanga inutilizzata e potrebbe cederla.(ne parleremo dopo). L acpiu istruira attraverso il dirver il controller per esaudire la richiesta, e deve aspettare l'esito della ricietsa, dove era la testina o alrro, che richiede un po' di tempo, e non è accettabile che il SO rimanga fermo li ad attendere. Il dialetto del controller avrà un suo modo per dare la risposta. il so deve chiedere quanto arriva adf ogni porta. per fare questo controllo il sistema operativo deve usare la cpu. c'è un modo per accorgersi dell'esito della operazione senza usare cpu, ed ecco che nascono gli interrupt, quindi i controller segnalano alla cpu uno degli interrupt disponibili.

cerca spiegazione slide 16 e controlla

direct memory access DMA cercare funzione x sistema operativo

overhead di un sistema